CTF CHEATSHEET

Cryptography & Steganography

Easy to Advanced Level

Complete Walkthrough with Commands & Scripts

>> Full Step-by-Step Walkthroughs for Every Tool

>> Real CTF Scenario Examples with Expected Outputs

>> Working Python Scripts for Every Attack

>> Beginner to Competition-Level Coverage

>> Open-Source Tools with Installation Guides

Generated for CTF Competitions

Chapter 1: Encoding & Decoding

Encoding is the most fundamental concept in CTFs. Many challenges use multiple layers of encoding. Always try decoding first before attempting complex crypto attacks.

Base64 / Base32 / Base16 Encoding

[Difficulty: Easy]

Base64 encodes binary data using 64 ASCII characters (A-Z, a-z, 0-9, +, /). Padding uses '=' characters. Base32 uses A-Z and 2-7. Base16 is just hex.

How It Works

Base64: Takes 3 bytes (24 bits), splits into 4 groups of 6 bits, maps each to a character. If input isn't multiple of 3, padding (=) is added. Base32: Takes 5 bytes, splits into 8 groups of 5 bits. Base16: Each byte becomes 2 hex digits.

How to Identify

- Base64: Characters A-Z, a-z, 0-9, +, / with = or == padding at end
- Base32: Uppercase A-Z and 2-7, padding with = (up to 6 = signs)
- Base16/Hex: Characters 0-9, A-F (case insensitive), always even length
- Base58: Like Base64 but no 0, O, I, l, +, / (used in Bitcoin addresses)
- Base85/ASCII85: Printable ASCII chars, often wrapped in <~ and ~>

WALKTHROUGH: Decoding Base64 - Real CTF Example

Step 1: You receive a file with this content:

```
Q1RGe2Jhc2U2NF9pc19lYXN5fQ==
```

Step 2: Recognize Base64: A-Z,a-z,0-9 chars + == padding. Decode it:

```
$ echo 'Q1RGe2Jhc2U2NF9pc19lYXN5fQ==' | base64 -d
CTF{base64_is_easy}
```

Step 3: If first decode gives another encoded string, keep decoding:

```
$ echo 'VTBaU1EyRnNNSGREWjFwSFZsZFdSMVpZU2pCaFZ6bDJXa2hLZWxwWE5XcGtSemw1' | base64 -d
U0ZSQ2FsMHdDZ1pHVldWR1ZYSG...
$ echo '<result>' | base64 -d   # keep going until readable text
```

Step 4: In Python for automation:

```
import base64
data = 'Q1RGe2Jhc2U2NF9pc19lYXN5fQ=='
print(base64.b64decode(data).decode())  # -> CTF{base64_is_easy}
```

CLI Commands - All Base Encodings

```
# Base64 decode/encode
echo 'SGVsbG8gV29ybGQ=' | base64 -d          # -> Hello World
echo -n 'Hello World' | base64                 # -> SGVsbG8gV29ybGQ=
base64 -d encoded_file.txt > decoded.bin       # decode file

# Base32 decode/encode
echo 'JBSWY3DPEBLW64TMMQ======' | base32 -d   # -> Hello World
echo -n 'Hello World' | base32

# Base16 (hex) decode
echo '48656c6c6f20576f726c64' | xxd -r -p      # -> Hello World
echo -n 'Hello World' | xxd -p                 # -> encode to hex

# Base58 (need python)
python3 -c "import base58; print(base58.b58decode('JxF12TrwUP45BMd').decode())"

# Base85 / ASCII85
```

```
python3 -c "import base64; print(base64.b85decode('87cURD]j7BEbo80').decode())"
```

Python - Auto Multi-Layer Decoder (IMPORTANT SCRIPT)

```
import base64, codecs
from urllib.parse import unquote

def auto_decode(s, depth=10, path=""):
    """Automatically tries all common decodings recursively"""
    if depth == 0:
        return
    if isinstance(s, bytes):
        try: s = s.decode('utf-8')
        except: return

    # Check if we found a flag
    if 'flag{' in s.lower() or 'ctf{' in s.lower():
        print(f"\n*** FLAG FOUND: {s} ***")
        print(f"    Path: {path}")
        return s

    decoders = [
        ("base64", lambda x: base64.b64decode(x + '==')),
        ("base32", lambda x: base64.b32decode(x + '======')),
        ("hex",    lambda x: bytes.fromhex(x.replace(' ',''))),
        ("rot13",  lambda x: codecs.decode(x, 'rot_13').encode()),
        ("url",    lambda x: unquote(x).encode()),
    ]
    for name, func in decoders:
        try:
            result = func(s)
            decoded = result.decode('utf-8', errors='strict')
            if decoded != s and len(decoded) > 2:
                printable = sum(1 for c in decoded if c.isprintable()) / len(decoded)
                if printable > 0.8:
                    print(f"{path}{name} -> {decoded[:80]}")
                    found = auto_decode(decoded, depth-1, path+name+" -> ")
                    if found:
                        return found
        except:
            pass

# USAGE:
auto_decode("YOUR_ENCODED_DATA_HERE")
```

TOOL: CyberChef

Browser-based encoding/decoding Swiss Army knife. Use 'Magic' recipe to auto-detect.

```
$ URL: https://gchq.github.io/CyberChef/
$ Key Recipes: From Base64, From Hex, From Binary, Magic, ROT13
```

WALKTHROUGH: Using CyberChef to Decode Unknown Data

Step 1: Open CyberChef in browser:

```
https://gchq.github.io/CyberChef/
```

Step 2: Paste encoded data into the 'Input' box on the right

Step 3: Drag 'Magic' from Operations (left panel) into Recipe (middle)

```
CyberChef will auto-detect encoding and show decoded output
```

Step 4: If Magic fails, try manually: drag 'From Base64' into Recipe

```
If output looks encoded again, drag another decoder below it
```

Step 5: For complex multi-layer: stack operations top-to-bottom

```
Example: From Base64 -> From Hex -> ROT13 -> Output shows flag
```

Hexadecimal Encoding

[Difficulty: Easy]

Hex represents each byte as two characters (0-9, A-F). Very common in CTFs for representing binary data, keys, ciphertexts.

WALKTHROUGH: Working with Hex Data

Step 1: You see this string: 666c61677b6865785f69735f66756e7d

```
All chars are 0-9 and a-f, even length = probably hex
```

Step 2: Decode with xxd:

```
$ echo '666c61677b6865785f69735f66756e7d' | xxd -r -p
flag{hex_is_fun}
```

Step 3: Decode with Python:

```
python3 -c "print(bytes.fromhex('666c61677b6865785f69735f66756e7d').decode())"
```

Step 4: To hex-dump a binary file:

```
$ xxd mystery.bin | head -20
00000000: 8950 4e47 0d0a 1a0a ...  .PNG....
This shows it's a PNG file (89 50 4E 47 header)
```

Hex Commands

```
# Decode hex string to text
echo '48656c6c6f' | xxd -r -p
python3 -c "print(bytes.fromhex('48656c6c6f').decode())"

# Encode text to hex
echo -n 'Hello' | xxd -p
python3 -c "print('Hello'.encode().hex())"

# Full hex dump of file (shows offset, hex, ASCII)
xxd file.bin                    # standard hex dump
xxd -g 1 file.bin               # group by single bytes
xxd -s 0x100 -l 64 file.bin     # start at offset 0x100, show 64 bytes
hexdump -C file.bin             # alternative hex dump tool
```

Binary, Octal & Decimal Encoding

[Difficulty: Easy]

WALKTHROUGH: Decoding Binary to Text

Step 1: You see: 01100110 01101100 01100001 01100111

```
Groups of 8 bits (0s and 1s) = binary encoding
```

Step 2: Each 8-bit group is one ASCII character:

```
01100110 = 102 = 'f'
01101100 = 108 = 'l'
01100001 = 97  = 'a'
01100111 = 103 = 'g'  -> 'flag'
```

Step 3: Decode with Python:

```
binary = '01100110 01101100 01100001 01100111'
text = ''.join(chr(int(b,2)) for b in binary.split())
print(text)  # flag
```

Python - Binary/Octal/Decimal Decoder

```
# Binary to text
binary = "01001000 01100101 01101100 01101100 01101111"
text = ''.join(chr(int(b, 2)) for b in binary.split())
print(f"Binary -> {text}")  # Hello

# Decimal (ASCII values) to text
decimal = "72 101 108 108 111"
text = ''.join(chr(int(d)) for d in decimal.split())
```

```
print(f"Decimal -> {text}")  # Hello

# Octal to text
octal_str = "110 145 154 154 157"
text = ''.join(chr(int(o, 8)) for o in octal_str.split())
print(f"Octal -> {text}")  # Hello

# How to identify:
# Binary: only 0s and 1s, groups of 8
# Octal: digits 0-7 only, groups of 3
# Decimal: numbers 32-126 for printable ASCII
```

Morse Code

[Difficulty: Easy]

Dots (.) and dashes (-) separated by spaces (letters) and / or | (words). Can appear as text, audio beeps, or visual blinks.

WALKTHROUGH: Decoding Morse Code

Step 1: You see: .... . .-.. .-.. --- / .-- --- .-. .-.. -..

```
Dots and dashes with spaces and / = Morse code
```

Step 2: Use the Morse alphabet to decode each letter:

```
.... = H, . = E, .-.. = L, .-.. = L, --- = O
/ = space
.-- = W, --- = O, .-. = R, .-.. = L, -.. = D
Result: HELLO WORLD
```

Step 3: Online tool for instant decode:

```
https://morsecode.world/international/translator.html
Paste morse code -> get text instantly
```

Step 4: If Morse is in audio form:

```
Open in Audacity -> View waveform -> short beep=dot, long beep=dash
```

Python Morse Decoder

```
MORSE = {'.-':'A','-...':'B','-.-.':'C','-..':'D','.':'E',
    '..-.':'F','--.':'G','....':'H','..':'I','.---':'J',
    '-.-':'K','.-..':'L','--':'M','-.':'N','---':'O',
    '.--.':'P','--.-':'Q','.-.':'R','...':'S','-':'T',
    '..-':'U','...-':'V','.--':'W','-..-':'X','-.--':'Y',
    '--..':'Z','-----':'0','.----':'1','..---':'2',
    '...--':'3','....-':'4','.....':'5','-....':'6',
    '--...':'7','---..':'8','----.':'9'}

msg = ".... . .-.. .-.. --- / .-- --- .-. .-.. -.."
words = msg.split(" / ")
decoded = ' '.join(''.join(MORSE.get(c,'?') for c in w.split()) for w in words)
print(decoded)  # HELLO WORLD
```

URL & HTML Encoding

[Difficulty: Easy]

WALKTHROUGH: URL Decode Example

Step 1: You see: %66%6C%61%67%7B%75%72%6C%5F%65%6E%63%6F%64%65%7D

```
Percent signs followed by hex pairs = URL encoding
```

Step 2: Decode:

```
python3 -c "from urllib.parse import unquote; print(unquote('%66%6C%61%67%7B%75%72%6C%5F%65%6E%63%6F%64%65%7
Result: flag{url_encode}
```

Step 3: Double URL encoding (common trick):

```
%2566%256C%2561%2567 -> first decode: %66%6C%61%67 -> second decode: flag
```

URL/HTML Decode Commands

```
# URL decode
python3 -c "from urllib.parse import unquote; print(unquote('%48%65%6C%6C%6F'))"
# URL encode
python3 -c "from urllib.parse import quote; print(quote('Hello World!'))"
# HTML entities decode
python3 -c "import html; print(html.unescape('&#72;&#101;&#108;&#108;&#111;'))"
# Double URL decode
python3 -c "from urllib.parse import unquote; print(unquote(unquote('%2548%2565%256c%256c%256f')))"
```

Multi-Layer Encoding Strategy

[Difficulty: Medium]

Many CTF challenges nest multiple encodings. Key: always check if decoded output looks like another encoding.

TIP: Try CyberChef 'Magic' first. If that fails, use the auto_decode Python script above.

TIP: Common nesting: Base64 -> Hex -> ROT13 -> Binary -> flag

Chapter 2: Classical Ciphers

Classical ciphers are the backbone of easy-to-medium CTF challenges. Frequency analysis and pattern recognition are your main weapons.

Caesar Cipher / ROT-N

[Difficulty: Easy]

Shifts each letter by N positions in the alphabet. ROT13 is the most common variant (shift=13). There are only 25 possible shifts, so brute force is trivial.

How It Works

Each letter is replaced: A->D, B->E, C->F (shift 3). Wraps around: X->A, Y->B, Z->C. Numbers and symbols are NOT changed. ROT13: A->N, B->O, etc. ROT13 is its own inverse (apply twice = original).

WALKTHROUGH: Caesar Cipher - Complete Walkthrough

Step 1: You receive ciphertext: 'Khoor Zruog'

```
Looks like English but shifted. Try Caesar brute force.
```

Step 2: Run brute force - try all 25 shifts:

```
Shift  0: Khoor Zruog
Shift  1: Jgnnq Yqtnf
Shift  2: Ifmmp Xpsme
Shift  3: Hello World  <-- READABLE!
...
```

Step 3: Found it! Shift 3 gives 'Hello World'

```
The key was shift=3 (classic Caesar)
```

Step 4: For ROT13 specifically:

```
$ echo 'Uryyb' | tr 'A-Za-z' 'N-ZA-Mn-za-m'
Hello
```

Python - Caesar Brute Force (25 shifts)

```
def caesar_brute(ciphertext):
    """Try all 25 shifts and print results"""
    for shift in range(26):
        decoded = ""
        for c in ciphertext:
            if c.isalpha():
                base = ord('A') if c.isupper() else ord('a')
                decoded += chr((ord(c) - base - shift) % 26 + base)
            else:
                decoded += c
        print(f"Shift {shift:2d}: {decoded}")

caesar_brute("Khoor Zruog")
# Output: Shift 3: Hello World
```

CLI One-Liners

```
# ROT13 (shift 13)
echo 'Uryyb Jbeyq' | tr 'A-Za-z' 'N-ZA-Mn-za-m'   # Hello World
python3 -c "import codecs; print(codecs.decode('Uryyb', 'rot_13'))"

# ROT47 (includes numbers and symbols)
echo 'w6==@' | tr '!-~' 'P-~!-O'

# Any ROT-N in Python
python3 -c "
```

```
ct='Khoor'; n=3
print(''.join(chr((ord(c)-65-n)%26+65) if c.isupper() else
              chr((ord(c)-97-n)%26+97) if c.islower() else c for c in ct))
"
```

TOOL: dcode.fr/caesar-cipher

Auto brute force all shifts and detect English.

```
$ https://www.dcode.fr/caesar-cipher
$ Paste text -> auto solves. Also handles ROT-N variants.
```

Vigenere Cipher

[Difficulty: Medium]

Polyalphabetic cipher: each letter uses a different Caesar shift based on a keyword. Key repeats cyclically. Much harder than Caesar because frequency analysis on the whole text doesn't work directly.

How It Works

Keyword 'KEY': K=10, E=4, Y=24. Plaintext 'HELLO': H+K=R, E+E=I, L+Y=J, L+K=V, O+E=S. Ciphertext: 'RIJVS'. To decrypt, subtract key shifts.

WALKTHROUGH: Vigenere - Complete Attack Walkthrough

Step 1: You have ciphertext that looks random but has some patterns

```
Use Index of Coincidence (IoC) to find key length
```

Step 2: Step A - Find key length:

```
Calculate IoC for key lengths 1-20.
If IoC > 0.06 for a key length, that's likely correct.
Alternative: Kasiski examination - find repeated sequences,
the GCD of distances between them suggests key length.
```

Step 3: Step B - Once you know key length (say 4):

```
Split ciphertext into 4 groups:
Group 1: every 4th letter starting at pos 0
Group 2: every 4th letter starting at pos 1
...
```

Step 4: Step C - Each group is a simple Caesar cipher:

```
Do frequency analysis on each group separately.
Most common letter in group = probably E (or T,A)
The shift tells you that key letter.
```

Step 5: Step D - Combine key letters:

```
If shifts are 10,4,24 -> key letters K,E,Y -> key is 'KEY'
```

Step 6: Or just use the automatic solver:

```
https://www.guballa.de/vigenere-solver
https://www.dcode.fr/vigenere-cipher
```

Python - Vigenere Decrypt + Key Length Finder

```
def vigenere_decrypt(ciphertext, key):
    result = []
    key_idx = 0
    for c in ciphertext:
        if c.isalpha():
            shift = ord(key[key_idx % len(key)].upper()) - ord('A')
            base = ord('A') if c.isupper() else ord('a')
            result.append(chr((ord(c) - base - shift) % 26 + base))
            key_idx += 1
        else:
            result.append(c)
    return ''.join(result)

def find_key_length(ct, max_len=20):
    """Use Index of Coincidence to find likely key lengths"""
```

```
    from collections import Counter
    text = ''.join(c.upper() for c in ct if c.isalpha())
    for kl in range(1, max_len+1):
        iocs = []
        for i in range(kl):
            group = text[i::kl]
            n = len(group)
            if n < 2: continue
            freq = Counter(group)
            ioc = sum(f*(f-1) for f in freq.values()) / (n*(n-1))
            iocs.append(ioc)
        avg_ioc = sum(iocs)/len(iocs) if iocs else 0
        marker = " ** LIKELY **" if avg_ioc > 0.06 else ""
        print(f"Key length {kl:2d}: IoC = {avg_ioc:.4f}{marker}")

# USAGE:
find_key_length("YOUR_CIPHERTEXT")
print(vigenere_decrypt("RIJVS", "KEY"))  # HELLO
```

TOOL: dcode.fr/vigenere-cipher

Auto key detection and decryption.

```
$ https://www.dcode.fr/vigenere-cipher
$ https://www.guballa.de/vigenere-solver  (excellent auto-solver)
```

Simple Substitution Cipher

[Difficulty: Medium]

Each letter maps to a fixed different letter (A->Q, B->W, etc). 26! possible keys = can't brute force. Use frequency analysis + word patterns.

WALKTHROUGH: Substitution Cipher Walkthrough

Step 1: You have: 'GSV UOZT RH HVXIVG'

```
Looks like substitution. Run frequency analysis.
```

Step 2: Use quipqiup.com automatic solver:

```
Go to https://quipqiup.com/
Paste 'GSV UOZT RH HVXIVG'
Click Solve
Result: 'THE FLAG IS SECRET' (Atbash cipher variant)
```

Step 3: Manual approach - frequency analysis:

```
Count letter frequencies in ciphertext.
Most common letter probably = E (12.7% in English)
2nd most = T (9.1%), then A (8.2%), O (7.5%)...
```

Step 4: Pattern matching:

```
3-letter word 'GSV' = probably 'THE'
This gives G->T, S->H, V->E
Substitute and look for more patterns
```

Step 5: Python frequency analysis:

```
from collections import Counter
freq = Counter(c for c in cipher if c.isalpha())
for letter, count in freq.most_common():
    print(f'{letter}: {count}')
```

Python - Frequency Analysis

```
from collections import Counter

def freq_analysis(text):
    letters = [c.upper() for c in text if c.isalpha()]
    freq = Counter(letters)
    total = len(letters)
    print("Letter Frequency Analysis:")
    for letter, count in freq.most_common():
```

```
        bar = '#' * int(count/total * 100)
        pct = count/total*100
        print(f"  {letter}: {count:4d} ({pct:5.1f}%) {bar}")
    print(f"\nEnglish freq order: E T A O I N S H R D L C U M W F G Y P B V K J X Q Z")

freq_analysis("YOUR CIPHERTEXT HERE")
```

TOOL: quipqiup.com

Best automatic substitution cipher solver. Uses word patterns.

```
$ https://quipqiup.com/ - Paste ciphertext, click Solve
$ https://www.dcode.fr/monoalphabetic-substitution - Alternative
```

Affine Cipher

[Difficulty: Medium]

E(x) = (ax + b) mod 26, D(x) = a_inv * (x - b) mod 26. 'a' must be coprime with 26 (valid a: 1,3,5,7,9,11,15,17,19,21,23,25).

WALKTHROUGH: Affine Cipher Attack

Step 1: You know it's an affine cipher (challenge hint)

```
Only 12 valid values of a, 26 values of b = 312 combinations
```

Step 2: Brute force all valid (a,b) pairs:

```
For each (a,b): decrypt and check if output contains English words
```

Step 3: If you know two plaintext-ciphertext pairs:

```
Solve: c1 = a*p1 + b mod 26, c2 = a*p2 + b mod 26
Subtract: (c1-c2) = a*(p1-p2) mod 26 -> solve for a
Then: b = c1 - a*p1 mod 26
```

Python - Affine Brute Force

```
from math import gcd

def affine_decrypt(ct, a, b):
    a_inv = pow(a, -1, 26)
    result = ''
    for c in ct:
        if c.isalpha():
            base = ord('A') if c.isupper() else ord('a')
            x = ord(c) - base
            result += chr((a_inv * (x - b)) % 26 + base)
        else:
            result += c
    return result

def affine_brute(ct):
    for a in range(1, 26):
        if gcd(a, 26) != 1: continue
        for b in range(26):
            pt = affine_decrypt(ct, a, b)
            # Check for common English words
            low = pt.lower()
            if any(w in low for w in ['the','flag','ctf','and','is ']):
                print(f"a={a:2d}, b={b:2d}: {pt}")

affine_brute("CIPHERTEXT_HERE")
```

Rail Fence / Zigzag Cipher

[Difficulty: Easy-Medium]

Text written in zigzag across N rails, then read row by row. Only the number of rails is the key.

WALKTHROUGH: Rail Fence Decode

Step 1: 'HOLELWRD OL' with 2 rails:

```
Rail 1: H O L E L
Rail 2:  W R D  O L
Read zigzag: H W O R L D   O L L E -> 'HELLO WORLD' (rearranged)
```

Step 2: Brute force: try rails 2 through 10:

```
Usually 2-5 rails. Check each for readable text.
```

Step 3: Use dcode.fr:

```
https://www.dcode.fr/rail-fence-cipher
Auto-tries all rail counts
```

Python - Rail Fence Brute Force

```
def rail_fence_decrypt(cipher, num_rails):
    n = len(cipher)
    pattern = list(range(num_rails)) + list(range(num_rails-2, 0, -1))
    indices = [[] for _ in range(num_rails)]
    for i in range(n):
        indices[pattern[i % len(pattern)]].append(i)
    result = [''] * n
    pos = 0
    for rail in range(num_rails):
        for idx in indices[rail]:
            result[idx] = cipher[pos]
            pos += 1
    return ''.join(result)

# Brute force all rail counts
ct = "YOUR_CIPHERTEXT"
for rails in range(2, 10):
    decoded = rail_fence_decrypt(ct, rails)
    print(f"Rails={rails}: {decoded}")
```

Other Classical Ciphers - Quick Reference

Cipher	How to Identify	How to Solve
Atbash	A=Z, B=Y reversed alphabet	dcode.fr/atbash-cipher
Bacon	Groups of 5 A/B letters	dcode.fr/bacon-cipher
Polybius	Pairs of digits 1-5	dcode.fr/polybius-cipher
Playfair	Even-length, digraph pairs	dcode.fr/playfair-cipher
Hill	Numbers, matrix-based	dcode.fr/hill-cipher
Beaufort	Like Vigenere variant	dcode.fr/beaufort-cipher
Tap Code	Pairs of taps/knocks	dcode.fr/tap-cipher
Book Cipher	Page/line/word numbers	Need the book as key
Enigma	Military rotor machine	cryptii.com/enigma

TIP: Unknown cipher? Try: dcode.fr/cipher-identifier or boxentriq.com/code-breaking/cipher-identifier

Chapter 3: XOR Encryption

XOR is extremely common in CTFs. Key property: A XOR B XOR B = A (self-inverse). If you know part of plaintext, you can recover the key. If key repeats, you can find key length and break it.

Single-Byte XOR

[Difficulty: Easy]

How It Works

Every byte of plaintext is XORed with the same single byte (0-255). Only 256 possible keys, so brute force is instant. The correct key produces readable English text.

WALKTHROUGH: Single-Byte XOR - Complete Example

Step 1: You have hex-encoded data:

```
1b37373331363f78151b7f2b783431333d78397828372d363c78373e783a393b3736
```

Step 2: Convert hex to bytes, then try all 256 XOR keys:

```
for key in range(256):
    decoded = bytes(b ^ key for b in data)
    if decoded looks like English text: FOUND IT
```

Step 3: Score each result by English character frequency:

```
Count letters like e, t, a, o, i, n, s, h, r, space
Highest scoring key = most likely correct
```

Step 4: Result with key 0x58 (88, 'X'):

```
Cooking MC's like a pound of bacon
```

Step 5: How to spot XOR in challenges:

```
- Hex/binary data that doesn't decode as any standard encoding
- Challenge mentions 'XOR' or 'exclusive or'
- Given a 'key' value or single character
```

Python - Single Byte XOR Brute Force

```
def single_byte_xor_brute(hex_data):
    data = bytes.fromhex(hex_data)

    # English letter frequency scoring
    eng_freq = {' ':13, 'e':12.7, 't':9.1, 'a':8.2, 'o':7.5,
                'i':7, 'n':6.7, 's':6.3, 'h':6.1, 'r':6}

    results = []
    for key in range(256):
        decoded = bytes([b ^ key for b in data])
        try:
            text = decoded.decode('ascii')
            score = sum(eng_freq.get(c.lower(), 0) for c in text)
            results.append((key, text, score))
        except:
            pass

    results.sort(key=lambda x: x[2], reverse=True)
    for key, text, score in results[:3]:
        key_char = chr(key) if 31 < key < 127 else '?'
        print(f"Key 0x{key:02x} ('{key_char}') Score={score:.0f}: {text[:80]}")

single_byte_xor_brute("1b37373331363f78151b7f2b783431333d78397828372d363c78373e783a393b3736")
```

Multi-Byte XOR with Known Plaintext

[Difficulty: Medium]

WALKTHROUGH: Known Plaintext XOR Attack

Step 1: You know the file starts with a PNG header (89 50 4E 47 0D 0A 1A 0A)

```
XOR encrypted_bytes XOR known_plaintext = key_bytes
```

Step 2: XOR first 8 bytes of ciphertext with PNG header:

```
key_fragment = encrypted[:8] XOR b'\x89PNG\r\n\x1a\n'
This gives you 8 bytes of the key
```

Step 3: If key is shorter than 8 bytes, you might have the full key

```
Try key lengths 1-8. If key_fragment[:4] == key_fragment[4:8],
then key is probably 4 bytes
```

Step 4: Decrypt entire file with recovered key:

```
plaintext = XOR(ciphertext, repeating_key)
Save as .png and open it
```

Python - Known Plaintext XOR

```
def xor_bytes(data, key):
    return bytes([data[i] ^ key[i % len(key)] for i in range(len(data))])

# SCENARIO: XOR'd PNG file
with open('encrypted.bin', 'rb') as f:
    encrypted = f.read()

# PNG header magic bytes
known_header = b'\x89PNG\r\n\x1a\n'

# Recover key fragment
key_fragment = xor_bytes(encrypted[:len(known_header)], known_header)
print(f"Key fragment (hex): {key_fragment.hex()}")
print(f"Key fragment (ASCII): {key_fragment.decode(errors='replace')}")

# Try different key lengths
for key_len in range(1, len(key_fragment)+1):
    key = key_fragment[:key_len]
    result = xor_bytes(encrypted, key)
    # Check if result starts with PNG header
    if result[:4] == b'\x89PNG':
        print(f"Key length {key_len} works! Key: {key.hex()}")
        with open('decrypted.png', 'wb') as f:
            f.write(result)
        break
```

Repeating Key XOR (Unknown Key)

[Difficulty: Medium-Hard]

How It Works

Key repeats: 'SECRET' XOR 'HELLOWORLD' = H^S, E^E, L^C, L^R, O^E, W^T, O^S, R^E, L^C, D^R. To break: 1) Find key length using Hamming distance, 2) Split into single-byte-XOR groups, 3) Solve each group independently.

WALKTHROUGH: Breaking Repeating XOR - Step by Step

Step 1: Step 1: Find key length using Hamming distance

```
For each candidate key size KS:
  Take first 4 blocks of KS bytes
  Calculate normalized Hamming distance between them
  Lowest distance = most likely key size
```

Step 2: Step 2: Split ciphertext by key position

```
If key size = 3:
  Group A: bytes at positions 0, 3, 6, 9...
  Group B: bytes at positions 1, 4, 7, 10...
  Group C: bytes at positions 2, 5, 8, 11...
```

Step 3: Step 3: Each group is single-byte XOR

```
Solve Group A -> key byte 0
Solve Group B -> key byte 1
Solve Group C -> key byte 2
Key = [byte0, byte1, byte2]
```

Step 4: Step 4: Decrypt with recovered key

```
plaintext = xor(ciphertext, key)
print(plaintext)
```

Python - Full Repeating XOR Attack

```
def hamming_distance(b1, b2):
    return sum(bin(a ^ b).count('1') for a, b in zip(b1, b2))

def find_key_size(data, min_ks=2, max_ks=40):
    scores = []
    for ks in range(min_ks, min(max_ks+1, len(data)//4)):
        blocks = [data[i*ks:(i+1)*ks] for i in range(4)]
        dists = []
        for i in range(len(blocks)):
            for j in range(i+1, len(blocks)):
                dists.append(hamming_distance(blocks[i], blocks[j]) / ks)
        scores.append((ks, sum(dists)/len(dists)))
    scores.sort(key=lambda x: x[1])
    print("Top key sizes (lower = more likely):")
    for ks, score in scores[:5]:
        print(f"  Key size {ks:2d}: score {score:.3f}")
    return [s[0] for s in scores[:5]]

def break_repeating_xor(data):
    key_sizes = find_key_size(data)
    eng_freq = {' ':13,'e':12.7,'t':9.1,'a':8.2,'o':7.5,'i':7,'n':6.7}

    for ks in key_sizes[:3]:
        key = bytearray()
        for i in range(ks):
            block = bytes(data[j] for j in range(i, len(data), ks))
            best_score, best_byte = 0, 0
            for b in range(256):
                decoded = bytes([c ^ b for c in block])
                score = sum(eng_freq.get(chr(c).lower(),0) for c in decoded if c<128)
                if score > best_score:
                    best_score, best_byte = score, b
            key.append(best_byte)
        result = bytes([data[i]^key[i%ks] for i in range(len(data))])
        print(f"\nKey size {ks}, Key: '{key.decode(errors='replace')}'")
        print(f"Decrypted: {result[:150].decode(errors='replace')}")

# USAGE:
import base64
data = base64.b64decode("YOUR_BASE64_DATA")
break_repeating_xor(data)
```

TOOL: xortool

Automated XOR analysis tool (finds key length and key).

```
$ pip install xortool
$ xortool encrypted.bin                          # auto-detect key length
$ xortool -l 6 encrypted.bin                     # specify key length
$ xortool -l 6 -c 20 encrypted.bin               # most frequent char = space (0x20)
```

WALKTHROUGH: Using xortool

Step 1: Install:

```
pip install xortool
```

Step 2: Run on encrypted file:

```
$ xortool encrypted.bin
The most probable key lengths:
   2:   17.4%
   4:   14.3%
   6:   31.2%  <-- most likely
Key-length can be 6*n
```

Step 3: Run with key length:

```
$ xortool -l 6 -c 20 encrypted.bin
# -c 20 means space (0x20) is most common char
Possible key: 'SECRET'
Decrypted text saved to xortool_out/
```

Step 4: Check output:

```
cat xortool_out/0.out  # view decrypted text
```

Chapter 4: RSA Cryptography

RSA is the most common public-key crypto in CTFs. Components: n=p*q (modulus), e (public exponent, usually 65537), d (private exponent), phi=(p-1)*(q-1). Encrypt: c = m^e mod n. Decrypt: m = c^d mod n.

RSA Basics - Small N Factoring

[Difficulty: Easy]

How RSA Works

1) Choose primes p, q. 2) n = p*q. 3) phi = (p-1)(q-1). 4) Choose e coprime to phi (usually 65537). 5) d = e^-1 mod phi. Public key: (n, e). Private key: d. Encrypt: c = m^e mod n. Decrypt: m = c^d mod n. Security relies on difficulty of factoring n into p and q.

WALKTHROUGH: RSA Small N - Complete Walkthrough

Step 1: Challenge gives you: n, e, c

```
n = 3233, e = 17, c = 2790
```

Step 2: Step 1: Factor n to find p and q

```
3233 = 61 x 53, so p=61, q=53
For small n: trial division, for larger: factordb.com
```

Step 3: Step 2: Calculate phi(n) = (p-1)(q-1)

```
phi = (61-1)(53-1) = 60 x 52 = 3120
```

Step 4: Step 3: Calculate d = e^-1 mod phi

```
d = pow(17, -1, 3120) = 2753
```

Step 5: Step 4: Decrypt: m = c^d mod n

```
m = pow(2790, 2753, 3233) = 65
chr(65) = 'A' or long_to_bytes(65) = b'A'
```

Python - Complete RSA Decrypt Script

```
from Crypto.Util.number import inverse, long_to_bytes, isPrime
import math

# === GIVEN VALUES ===
n = 0   # modulus (product of p * q)
e = 65537  # public exponent
c = 0   # ciphertext

# === METHOD 1: Factor small n by trial division ===
def factor_small(n):
    """Works for n up to ~25 digits"""
    for i in range(2, int(math.isqrt(n)) + 1):
        if n % i == 0:
            return i, n // i
    return None, None

p, q = factor_small(n)
if p:
    print(f"p = {p}")
    print(f"q = {q}")
    phi = (p - 1) * (q - 1)
    d = pow(e, -1, phi)  # Python 3.8+, or use inverse(e, phi)
    m = pow(c, d, n)
    plaintext = long_to_bytes(m)
    print(f"Plaintext: {plaintext}")

# === METHOD 2: Use factordb for larger n ===
# pip install factordb-pycli
from factordb.factordb import FactorDB
```

```
f = FactorDB(n)
f.connect()
factors = f.get_factor_list()
print(f"Factors from factordb: {factors}")
if len(factors) == 2:
    p, q = factors
    phi = (p-1) * (q-1)
    d = pow(e, -1, phi)
    m = pow(c, d, n)
    print(f"Plaintext: {long_to_bytes(m)}")
```

TOOL: factordb.com

Online database of known integer factorizations.

```
$ http://factordb.com/ - Enter n, click Factorize
$ pip install factordb-pycli  # Python API
```

WALKTHROUGH: Using factordb.com

Step 1: Go to http://factordb.com/

Step 2: Paste your n value into the text box

Step 3: Click 'Factorize!'

```
If status shows 'FF' = Fully Factored -> factors shown
If status shows 'C' = Composite, not yet factored
If 'C': try yafu, msieve, or CADO-NFS locally
```

Step 4: Copy p and q, use in Python script above

TOOL: RsaCtfTool

All-in-one RSA attack tool - tries 40+ attacks automatically.

```
$ git clone https://github.com/RsaCtfTool/RsaCtfTool
$ cd RsaCtfTool && pip install -r requirements.txt
$ python3 RsaCtfTool.py --publickey pub.pem --uncipherfile cipher.enc
$ python3 RsaCtfTool.py -n <N> -e <E> --uncipher <C>
```

WALKTHROUGH: Using RsaCtfTool

Step 1: Scenario 1: You have a public key file (pub.pem):

```
$ python3 RsaCtfTool.py --publickey pub.pem --uncipherfile flag.enc
Tool auto-tries: factordb, Wiener, Fermat, Hastad, etc.
Output: Decrypted text or saves decrypted file
```

Step 2: Scenario 2: You have n, e, c as numbers:

```
$ python3 RsaCtfTool.py -n 12345...  -e 65537 --uncipher 9876...
Same auto-attack process
```

Step 3: Scenario 3: Extract private key:

```
$ python3 RsaCtfTool.py --publickey pub.pem --private
Outputs private key in PEM format
```

Step 4: If auto fails, check output for 'attack' names tried

```
Then research specific attacks for your scenario
```

Wiener's Attack (Small d / Very Large e)

[Difficulty: Medium]

When the private exponent d is small (d < n^0.25 / 3), e will be unusually large. The continued fraction expansion of e/n reveals d. Red flag: e is almost as large as n.

WALKTHROUGH: Wiener's Attack

Step 1: Identify: e is very large (close to n in size)

```
Normal e = 65537. If e is huge -> try Wiener's
```

Step 2: Using owiener library:

```
pip install owiener
import owiener
d = owiener.attack(e, n)
if d: print(f'Found d = {d}')
```

Step 3: Then decrypt normally:

```
m = pow(c, d, n)
print(long_to_bytes(m))
```

Python - Wiener's Attack

```
# pip install owiener
import owiener
from Crypto.Util.number import long_to_bytes

e = 123456789...  # very large e
n = 987654321...
c = 111222333...  # ciphertext

d = owiener.attack(e, n)
if d:
    print(f"Wiener's attack succeeded! d = {d}")
    m = pow(c, d, n)
    print(f"Plaintext: {long_to_bytes(m)}")
else:
    print("Wiener's attack failed - d may not be small enough")
```

Hastad's Broadcast Attack (e=3, multiple recipients)

[Difficulty: Medium]

Same plaintext m encrypted with e=3 to 3 different recipients (different n values). CRT recovers m^3, then cube root gives m.

WALKTHROUGH: Hastad's Broadcast Attack

Step 1: Identify: e=3, same message sent to 3+ people

```
You have (n1,c1), (n2,c2), (n3,c3) all with e=3
```

Step 2: Use Chinese Remainder Theorem:

```
Find x such that x = c1 mod n1, x = c2 mod n2, x = c3 mod n3
```

Step 3: x = m^3 (no modular reduction because m^3 < n1*n2*n3)

```
Take integer cube root: m = x^(1/3)
```

Python - Hastad's Attack

```
from sympy.ntheory.modular import crt
from gmpy2 import iroot  # pip install gmpy2
from Crypto.Util.number import long_to_bytes

# Three recipients, same message, e=3
n1, n2, n3 = ...   # three different moduli
c1, c2, c3 = ...   # three ciphertexts (same plaintext)
e = 3

# Chinese Remainder Theorem
m_cubed, _ = crt([n1, n2, n3], [c1, c2, c3])

# Take e-th root
m, is_perfect = iroot(m_cubed, e)
if is_perfect:
    print(f"Plaintext: {long_to_bytes(int(m))}")
else:
```

```
    print("Not a perfect cube - attack may have failed")
```

Common Modulus Attack

[Difficulty: Medium]

Same plaintext, same n, but two different public exponents e1 and e2 where gcd(e1,e2)=1.

WALKTHROUGH: Common Modulus Attack

Step 1: Identify: same n, two different e values, same plaintext

```
You have (n, e1, c1) and (n, e2, c2)
```

Step 2: Find s1, s2 such that e1*s1 + e2*s2 = 1 (extended GCD)

```
Since gcd(e1,e2)=1, this is guaranteed to exist
```

Step 3: Compute: m = c1^s1 * c2^s2 mod n

```
This works because c1^s1 * c2^s2 = m^(e1*s1 + e2*s2) = m^1 = m
```

Python - Common Modulus Attack

```
from Crypto.Util.number import long_to_bytes
from math import gcd

def common_modulus_attack(n, e1, e2, c1, c2):
    assert gcd(e1, e2) == 1, "e1 and e2 must be coprime"

    def extended_gcd(a, b):
        if a == 0: return b, 0, 1
        g, x, y = extended_gcd(b % a, a)
        return g, y - (b//a)*x, x

    _, s1, s2 = extended_gcd(e1, e2)

    # Handle negative exponents with modular inverse
    c1_val = pow(c1, s1, n) if s1 >= 0 else pow(pow(c1, -1, n), -s1, n)
    c2_val = pow(c2, s2, n) if s2 >= 0 else pow(pow(c2, -1, n), -s2, n)

    m = (c1_val * c2_val) % n
    return long_to_bytes(m)

plaintext = common_modulus_attack(n, e1, e2, c1, c2)
print(f"Plaintext: {plaintext}")
```

Fermat Factoring (p and q are close)

[Difficulty: Medium]

If p and q are close to each other, n = p*q can be factored quickly by searching near sqrt(n).

WALKTHROUGH: Fermat Factoring

Step 1: When to use: challenge says primes are 'close' or 'similar'

```
If |p-q| is small, Fermat's method is fast
```

Step 2: Algorithm:

```
Start with a = ceil(sqrt(n))
b2 = a^2 - n
If b2 is a perfect square: p = a-b, q = a+b
Otherwise increment a and try again
```

Python - Fermat Factoring

```
from math import isqrt

def fermat_factor(n, max_iter=1000000):
    a = isqrt(n) + 1
```

```
    for _ in range(max_iter):
        b2 = a*a - n
        b = isqrt(b2)
        if b*b == b2:
            p, q = a-b, a+b
            assert p * q == n
            print(f"Factored! p={p}, q={q}")
            return p, q
        a += 1
    print("Fermat factoring failed - primes may not be close enough")
    return None, None

p, q = fermat_factor(n)
```

RSA Attack Selection Guide

What You See	Attack	Difficulty
Small n (<512 bits)	Factor with factordb/yafu/trial	Easy
p and q are close	Fermat factoring	Easy-Med
e is very large (close to n)	Wiener's attack	Medium
e=3, same msg to 3+ people	Hastad's broadcast (CRT)	Medium
Same n, different e values	Common modulus attack	Medium
e=3 and m^3 < n	Direct cube root	Easy
dp or dq leaked	Partial key recovery	Medium
Multiple keys share a factor	GCD of moduli	Easy
Known partial plaintext	Coppersmith's method	Hard
Related messages (same n,e)	Franklin-Reiter	Hard

Chapter 5: AES & Symmetric Crypto Attacks

AES operates on 16-byte blocks. The mode of operation (ECB, CBC, CTR) determines vulnerabilities. Key sizes: 128, 192, or 256 bits.

AES-ECB Detection & Attacks

[Difficulty: Easy-Medium]

How ECB Works

Each 16-byte block is encrypted independently with the same key. PROBLEM: same plaintext block always produces same ciphertext block. This leaks patterns.

WALKTHROUGH: ECB Detection and Exploitation

Step 1: Step 1: Detect ECB - look for repeated ciphertext blocks

```
Split ciphertext into 16-byte chunks.
If any chunks are identical -> ECB mode.
```

Step 2: Step 2: ECB byte-at-a-time oracle attack:

```
If you can prepend chosen text before a secret:
- Send 15 A's -> first block = AAAAAAAAAAAAAAAS (S=first secret byte)
- Try all 256 values for last byte until block matches
- Now you know first byte of secret. Send 14 A's to get second byte...
```

Step 3: Step 3: ECB cut-and-paste attack:

```
If you can control some plaintext:
- Craft input to align 'admin' at block boundary
- Capture that block's ciphertext
- Replace a block in another request with the 'admin' block
```

Python - ECB Detection

```
from collections import Counter

def detect_ecb(ciphertext):
    """Check if ciphertext uses ECB mode (repeated blocks)"""
    if isinstance(ciphertext, str):
        ciphertext = bytes.fromhex(ciphertext)

    blocks = [ciphertext[i:i+16] for i in range(0, len(ciphertext), 16)]
    unique = len(set(blocks))
    total = len(blocks)

    if unique < total:
        print(f"ECB DETECTED! {total} blocks, only {unique} unique")
        repeated = Counter(blocks)
        for block, count in repeated.most_common():
            if count > 1:
                print(f"  Block {block.hex()} appears {count} times")
        return True
    else:
        print(f"Probably NOT ECB ({total} blocks, all unique)")
        return False
```

AES-CBC Padding Oracle Attack

[Difficulty: Hard]

How It Works

CBC uses padding (PKCS7) to fill the last block. If the server tells you whether padding is valid or invalid (different error messages, timing differences), you can decrypt the entire ciphertext byte by byte WITHOUT knowing the key.

WALKTHROUGH: Padding Oracle Attack - Step by Step

Step 1: Concept: PKCS7 padding

```
If last block needs 3 bytes padding: ...\x03\x03\x03
If needs 1 byte: ...\x01
Server checks if padding is valid after decryption
```

Step 2: Step 1: Target the last byte of a block

```
Modify byte in PREVIOUS ciphertext block.
Try all 256 values until server says 'valid padding'.
When valid: you know the intermediate decryption value.
```

Step 3: Step 2: Work backwards byte by byte

```
After finding last byte, target second-to-last.
Set last byte to produce \x02 padding.
Brute force second-to-last to also produce \x02.
```

Step 4: Step 3: Repeat for all bytes, all blocks

```
Each block decrypted independently using previous block.
First block uses IV as 'previous block'.
```

Step 5: Automated tool:

```
$ padbuster http://target/api CIPHERTEXT_HEX 16
PadBuster auto-exploits padding oracle vulnerabilities
```

TOOL: PadBuster

Automated padding oracle exploitation tool.

```
$ padbuster URL CIPHER_HEX BLOCK_SIZE
$ padbuster http://target/path AABB...FF 16
$ padbuster http://target/path CIPHER 16 -encoding 0 -plaintext 'admin'
```

AES-CBC Bit Flipping Attack

[Difficulty: Medium]

In CBC mode, flipping a bit in ciphertext block N corrupts block N's plaintext but PREDICTABLY changes block N+1's plaintext. Used to modify encrypted data without knowing the key.

WALKTHROUGH: CBC Bit Flipping - Example

Step 1: Scenario: Cookie has encrypted 'role=user'

```
You want to change it to 'role=admin' without the key
```

Step 2: CBC property: plaintext[i] = decrypt(ct[i]) XOR ct[i-1]

```
To change plaintext byte, modify PREVIOUS ciphertext block
```

Step 3: Formula:

```
new_ct[pos] = old_ct[pos] XOR old_char XOR new_char
Example: to flip 'u' to 'a' at position X:
new_ct[X-16] = old_ct[X-16] XOR ord('u') XOR ord('a')
```

Step 4: Result: block containing the flip becomes garbled,

```
but the NEXT block has your desired modification
```

Python - CBC Bit Flip

```
def cbc_bit_flip(ciphertext, position, old_byte, new_byte, block_size=16):
    """Modify plaintext byte at 'position' in block N+1
    by flipping the corresponding byte in ciphertext block N"""
    ct = bytearray(ciphertext)
    # The byte to modify is in the PREVIOUS block
    modify_pos = position - block_size
    ct[modify_pos] ^= old_byte ^ new_byte
    return bytes(ct)
```

```
# Example: Change 'user' to 'admn' in encrypted cookie
# (assuming 'user' starts at byte 32 in plaintext)
modified = cbc_bit_flip(ciphertext, 32, ord('u'), ord('a'))
modified = cbc_bit_flip(modified,   33, ord('s'), ord('d'))
modified = cbc_bit_flip(modified,   34, ord('e'), ord('m'))
modified = cbc_bit_flip(modified,   35, ord('r'), ord('n'))
```

OpenSSL Symmetric Decryption Commands

```
# AES-128-CBC
openssl enc -aes-128-cbc -d -in encrypted.bin -out decrypted.txt -K <hex_key> -iv <hex_iv>

# AES-256-CBC with password
openssl enc -aes-256-cbc -d -in encrypted.bin -out decrypted.txt -pass pass:password

# DES-CBC
openssl enc -des-cbc -d -in encrypted.bin -out decrypted.txt -K <hex_key> -iv <hex_iv>

# 3DES
openssl enc -des-ede3-cbc -d -in enc.bin -out dec.txt -K <hex_key> -iv <hex_iv>

# List all available ciphers
openssl enc -list
```

Chapter 6: Hash Functions & Cracking

Hashes are one-way functions. CTF tasks: crack hashes (find input), exploit collisions, length extension attacks. Start by identifying the hash type, then try online lookup before brute force.

Hash Type Identification

[Difficulty: Easy]

Hash Type	Length	Example Start	Regex Pattern
MD5	32 hex	d41d8cd9...	^[a-f0-9]{32}$
SHA-1	40 hex	da39a3ee...	^[a-f0-9]{40}$
SHA-256	64 hex	e3b0c442...	^[a-f0-9]{64}$
SHA-512	128 hex	cf83e135...	^[a-f0-9]{128}$
bcrypt	60 chars	$2b$10$...	^\$2[aby]\$
NTLM	32 hex	aad3b435...	Same as MD5
MySQL 4.1+	41 chars	*6BB4837...	^*[A-F0-9]{40}$

WALKTHROUGH: Identifying and Cracking a Hash

Step 1: You have: 5f4dcc3b5aa765d61d8327deb882cf99

```
32 hex characters -> MD5 or NTLM
```

Step 2: Step 1: Identify hash type:

```
$ hashid '5f4dcc3b5aa765d61d8327deb882cf99'
[+] MD5
[+] NTLM
```

Step 3: Step 2: Try online lookup first (instant):

```
Go to https://crackstation.net
Paste hash -> Result: 'password'
This hash is MD5('password')
```

Step 4: Step 3: If online fails, use hashcat:

```
$ hashcat -m 0 hash.txt rockyou.txt
-m 0 = MD5 mode
rockyou.txt = common password wordlist
```

Step 5: Step 4: If wordlist fails, try rules:

```
$ hashcat -m 0 hash.txt rockyou.txt -r rules/best64.rule
Rules add mutations: capitalize, append numbers, etc.
```

TOOL: hashid / haiti

Auto-identify hash types.

```
$ pip install hashid
$ hashid '5f4dcc3b5aa765d61d8327deb882cf99'
$ pip install haiti
$ haiti '5f4dcc3b5aa765d61d8327deb882cf99'  # also shows hashcat mode
```

Hashcat - GPU Hash Cracking

[Difficulty: Easy-Medium]

WALKTHROUGH: Hashcat Complete Workflow

Step 1: Step 1: Identify hash mode number:

```
MD5 = 0, SHA1 = 100, SHA256 = 1400, bcrypt = 3200, NTLM = 1000
Full list: hashcat --help | grep -i 'hash-type'
```

Step 2: Step 2: Wordlist attack:

```
$ hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt
```

Tries every word in rockyou.txt

Step 3: Step 3: If wordlist fails, add rules:

```
$ hashcat -m 0 hash.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule
Rules: Password -> password, Password1, P@ssword, etc.
```

Step 4: Step 4: Mask (brute force) attack:

```
$ hashcat -m 0 hash.txt -a 3 ?l?l?l?l?l?l
?l = lowercase, ?u = upper, ?d = digit, ?s = special, ?a = all
flag{?a?a?a?a} for known format
```

Step 5: Step 5: Check results:

```
$ hashcat -m 0 hash.txt --show
5f4dcc3b...:password
```

Hashcat Command Reference

```
# Dictionary attack
hashcat -m 0 hash.txt rockyou.txt              # MD5
hashcat -m 100 hash.txt rockyou.txt            # SHA-1
hashcat -m 1400 hash.txt rockyou.txt           # SHA-256
hashcat -m 1000 hash.txt rockyou.txt           # NTLM
hashcat -m 3200 hash.txt rockyou.txt           # bcrypt (slow!)

# With rules
hashcat -m 0 hash.txt rockyou.txt -r rules/best64.rule

# Mask attack (brute force)
hashcat -m 0 hash.txt -a 3 ?a?a?a?a?a?a         # 6 chars all
hashcat -m 0 hash.txt -a 3 ?u?l?l?l?l?d?d        # Aaaaa00 pattern
hashcat -m 0 hash.txt -a 3 'flag{?a?a?a?a?a}'    # known format

# Combination attack (word1+word2)
hashcat -m 0 hash.txt -a 1 wordlist1.txt wordlist2.txt

# Show cracked hashes
hashcat -m 0 hash.txt --show
```

John the Ripper

[Difficulty: Easy-Medium]

WALKTHROUGH: John the Ripper Workflow

Step 1: Step 1: Auto-detect format:

```
$ john hash.txt
John auto-detects hash type and starts cracking
```

Step 2: Step 2: Specify format + wordlist:

```
$ john --format=raw-md5 --wordlist=rockyou.txt hash.txt
```

Step 3: Step 3: Show cracked passwords:

```
$ john --show hash.txt
username:password
```

Step 4: For protected files:

```
$ zip2john protected.zip > zip_hash.txt
$ john zip_hash.txt --wordlist=rockyou.txt
$ john --show zip_hash.txt
```

John Commands for Different Formats

```
# Auto-detect
john hash.txt

# Specific format + wordlist
john --format=raw-md5 --wordlist=rockyou.txt hash.txt
john --format=raw-sha256 --wordlist=rockyou.txt hash.txt
john --format=bcrypt --wordlist=rockyou.txt hash.txt
```

```
# Convert protected files to john format
zip2john protected.zip > zip_hash.txt    # ZIP files
rar2john protected.rar > rar_hash.txt    # RAR files
pdf2john protected.pdf > pdf_hash.txt    # PDF files
ssh2john id_rsa > ssh_hash.txt           # SSH keys
gpg2john key.gpg > gpg_hash.txt          # GPG keys
keepass2john db.kdbx > kp_hash.txt       # KeePass

# Then crack
john zip_hash.txt --wordlist=rockyou.txt

# Show results
john --show hash.txt
```

Online Hash Lookup (Try These First!)

[Difficulty: Easy]

- crackstation.net - Massive rainbow table, FREE (best first try)
- hashes.org - Community database of cracked hashes
- cmd5.org - MD5/SHA1/NTLM lookup
- hashtoolkit.com - Reverse hash lookup
- md5decrypt.net - MD5 decryption

TIP: ALWAYS try online lookup before running hashcat/john. Many CTF hashes are common passwords!

Hash Length Extension Attack

[Difficulty: Hard]

If MAC = H(secret || message) using MD5/SHA1/SHA256, you can compute H(secret || message || padding || extra_data) without knowing the secret.

WALKTHROUGH: Hash Length Extension

Step 1: Scenario: Server computes MAC = SHA256(secret + data)

```
You know: data, MAC, and length of secret (or can guess it)
```

Step 2: You want to compute SHA256(secret + data + padding + ';admin=true')

```
Without knowing the secret!
```

Step 3: Use hashpump:

```
pip install hashpumpy
import hashpumpy
new_hash, new_data = hashpumpy.hashpump(
    original_hash, original_data, ';admin=true', secret_length)
```

Step 4: Send new_data with new_hash to server

```
Server will verify MAC and accept the modified message
```

Python - Hash Length Extension

```
import hashpumpy  # pip install hashpumpy

original_hash = "6d5f807e23db210bc254a28be2d6759a0f5f5d99"
original_data = b"count=10&lat=37.351"
append_data = b"&admin=true"

# Try different secret lengths (1-32)
for secret_len in range(1, 33):
    new_hash, new_data = hashpumpy.hashpump(
        original_hash, original_data, append_data, secret_len
    )
    print(f"Secret len {secret_len:2d}: hash={new_hash[:20]}... data={new_data[:50]}")
```

Chapter 7: Advanced & Misc Cryptography

One-Time Pad / Many-Time Pad (Crib Dragging)

[Difficulty: Medium-Hard]

OTP is unbreakable IF key is used ONCE. If key is reused (many-time pad), XOR two ciphertexts: c1 XOR c2 = m1 XOR m2 (key cancels out). Then use 'crib dragging' to guess words.

WALKTHROUGH: Many-Time Pad / Crib Dragging Attack

Step 1: You have 2+ ciphertexts encrypted with the same key

```
c1 = encrypt(m1, key), c2 = encrypt(m2, key)
```

Step 2: Step 1: XOR the two ciphertexts together

```
c1 XOR c2 = m1 XOR m2 (key cancels!)
```

Step 3: Step 2: Drag a common word across the XOR result

```
Try 'the ', ' the', 'flag', etc. at each position.
XOR 'the ' with xored_data at position i.
If result looks like English -> you found part of the other message.
```

Step 4: Step 3: Build up both messages

```
As you find fragments, they reveal more of both messages.
Eventually recover enough to guess the rest.
```

Step 5: Tool: mtp

```
https://github.com/CameronLonsworthy/MTP
Interactive crib dragging with visual interface
```

Python - Crib Dragging Script

```
def crib_drag(c1_hex, c2_hex, crib):
    c1 = bytes.fromhex(c1_hex)
    c2 = bytes.fromhex(c2_hex)
    xored = bytes(a ^ b for a, b in zip(c1, c2))

    if isinstance(crib, str):
        crib = crib.encode()

    print(f"Dragging crib: '{crib.decode()}'")
    for i in range(len(xored) - len(crib) + 1):
        result = bytes(xored[i+j] ^ crib[j] for j in range(len(crib)))
        try:
            text = result.decode('ascii')
            if all(c.isprintable() or c == ' ' for c in text):
                print(f"  Position {i:3d}: '{text}'")
        except:
            pass

# XOR two ciphertexts and drag common English words
crib_drag("c1_hex_here", "c2_hex_here", "the ")
crib_drag("c1_hex_here", "c2_hex_here", "flag{")
crib_drag("c1_hex_here", "c2_hex_here", " is ")
```

PRNG Attacks - Mersenne Twister (MT19937)

[Difficulty: Hard]

Python's random module uses MT19937. If you observe 624 consecutive 32-bit outputs, you can clone the internal state and predict ALL future outputs.

WALKTHROUGH: Cracking Python's random

Step 1: Step 1: Collect 624 consecutive random.getrandbits(32) outputs

```
These can come from any function using random internally
```

Step 2: Step 2: Use randcrack to clone state:

```
from randcrack import RandCrack
rc = RandCrack()
for output in outputs_624:
    rc.feed(output)
```

Step 3: Step 3: Predict future outputs:

```
next_value = rc.predict_getrandbits(32)
next_randint = rc.predict_randint(0, 100)
```

Python - MT19937 Clone

```
# pip install randcrack
from randcrack import RandCrack

rc = RandCrack()

# Feed exactly 624 consecutive 32-bit random outputs
for i in range(624):
    rc.feed(observed_outputs[i])  # each is a 32-bit integer

# Now predict future outputs
next_val = rc.predict_getrandbits(32)
next_int = rc.predict_randint(0, 255)
next_random = rc.predict_random()  # float between 0 and 1
```

Chapter 8: Image Steganography

Image steganography hides data within image files. ALWAYS follow the systematic workflow below before trying specialized tools.

Initial Analysis Workflow (DO THIS FIRST!)

[Difficulty: Easy]

WALKTHROUGH: Step-by-Step First Analysis of Any Image

Step 1: Step 1: Check true file type

```
$ file suspicious.png
suspicious.png: PNG image data, 800 x 600, 8-bit/color RGB
(If file type doesn't match extension -> renamed file!)
```

Step 2: Step 2: Read ALL metadata

```
$ exiftool suspicious.png
Look for: Comment, Description, Author, GPS, custom fields
The flag might be right here in metadata!
```

Step 3: Step 3: Search for strings

```
$ strings suspicious.png | grep -iE 'flag|ctf|key|pass'
Also: strings -n 10 suspicious.png  (min 10 char strings)
```

Step 4: Step 4: Check file header in hex

```
$ xxd suspicious.png | head -5
Verify magic bytes match expected format
89 50 4E 47 = PNG, FF D8 FF = JPEG, 42 4D = BMP
```

Step 5: Step 5: Scan for embedded files

```
$ binwalk suspicious.png
Look for: ZIP, RAR, PNG, JPEG, ELF inside the image
$ binwalk -e suspicious.png  (extract embedded files)
```

Step 6: Step 6: Quick stego scans

```
For PNG/BMP: $ zsteg -a suspicious.png
For JPEG: $ steghide info suspicious.jpg
For PNG: $ pngcheck -v suspicious.png
```

Complete First-Analysis Script

```
#!/bin/bash
# save as: stego_check.sh
# Usage: ./stego_check.sh image_file

FILE=$1
echo "=== FILE TYPE ==="
file "$FILE"
echo ""

echo "=== EXIF DATA ==="
exiftool "$FILE"
echo ""

echo "=== STRINGS (flags/keys) ==="
strings "$FILE" | grep -iE 'flag|ctf|key|pass|secret|hidden'
echo ""

echo "=== BINWALK ==="
binwalk "$FILE"
echo ""

echo "=== HEX HEADER ==="
xxd "$FILE" | head -10
echo ""
```

```
echo "=== HEX TAIL ==="
xxd "$FILE" | tail -10
echo ""

# PNG-specific
if file "$FILE" | grep -q PNG; then
    echo "=== ZSTEG (PNG) ==="
    zsteg -a "$FILE" 2>/dev/null
    echo ""
    echo "=== PNGCHECK ==="
    pngcheck -v "$FILE" 2>/dev/null
fi

# JPEG-specific
if file "$FILE" | grep -q JPEG; then
    echo "=== STEGHIDE (JPEG) ==="
    steghide info "$FILE" 2>/dev/null
fi
```

exiftool - Metadata Analysis

[Difficulty: Easy]

TOOL: exiftool

Read/write/edit metadata in 100+ file formats.

```
$ sudo apt install libimage-exiftool-perl  # Linux install
```

WALKTHROUGH: Using exiftool

Step 1: Basic metadata read:

```
$ exiftool image.jpg
File Name: image.jpg
File Size: 2.3 MB
Image Size: 1920x1080
Comment: flag{metadata_is_easy}
  <-- FLAG IN COMMENT!
```

Step 2: Check specific fields:

```
$ exiftool -Comment image.jpg
$ exiftool -Description image.jpg
$ exiftool -Author image.jpg
$ exiftool -GPS* image.jpg  # GPS coordinates -> Google Maps
```

Step 3: Extract embedded thumbnail:

```
$ exiftool -b -ThumbnailImage image.jpg > thumb.jpg
Sometimes thumbnail contains different image with flag
```

Step 4: View ALL tags:

```
$ exiftool -all image.jpg
$ exiftool -v3 image.jpg  # verbose mode, shows raw data
```

LSB (Least Significant Bit) Steganography

[Difficulty: Easy-Medium]

How LSB Works

Each pixel has RGB values (0-255). The least significant bit (LSB) of each byte is changed to encode data. Changing LSB: 254->255 is visually imperceptible. Read LSBs in order to recover hidden message.

WALKTHROUGH: LSB Stego - Extraction Walkthrough

Step 1: Step 1: Try zsteg (for PNG/BMP):

```
$ zsteg -a image.png
b1,r,lsb,xy : 'flag{lsb_is_fun}'
b1,rgb,lsb,xy : PNG image data...
[*] Found something in bit 1, red channel, LSB, xy order
```

Step 2: Step 2: If zsteg finds data, extract it:

```
$ zsteg -e 'b1,rgb,lsb,xy' image.png > extracted.bin
This extracts the raw data from that specific channel/bit
```

Step 3: Step 3: For JPEG, use steghide:

```
$ steghide extract -sf image.jpg
Enter passphrase: (try empty first, press Enter)
Wrote extracted data to 'flag.txt'
```

Step 4: Step 4: If steghide needs password, brute force:

```
$ stegcracker image.jpg /usr/share/wordlists/rockyou.txt
Trying word 1/14344392: 'password' -> Found!
```

Step 5: Step 5: If tools fail, try Python manual extraction:

```
Extract LSBs from each pixel manually (script below)
```

TOOL: zsteg

LSB stego detector for PNG and BMP. Best automatic tool.

```
$ gem install zsteg                              # Install
$ zsteg image.png                                # Quick scan
$ zsteg -a image.png                             # Try ALL methods
$ zsteg -e 'b1,rgb,lsb,xy' image.png > out.bin  # Extract specific
$ zsteg -e 'b1,r,lsb,xy' image.png              # Red channel only
$ zsteg -e 'b2,rgb,lsb,xy' image.png             # 2nd bit plane
```

WALKTHROUGH: Understanding zsteg Output

Step 1: Output: b1,rgb,lsb,xy : 'flag{hidden}'

```
b1 = bit plane 1 (LSB)
rgb = all three color channels
lsb = least significant bit first
xy = left-to-right, top-to-bottom scan order
```

Step 2: Output: b1,r,lsb,xy : PNG image

```
Hidden PNG image in red channel LSB!
Extract: zsteg -e 'b1,r,lsb,xy' image.png > hidden.png
```

Step 3: Output: b2,g,msb,yx : 'data...'

```
b2 = 2nd bit plane, green channel, most significant bit, yx scan
```

TOOL: steghide

Hide/extract data in JPEG, BMP, WAV, AU files.

```
$ sudo apt install steghide                      # Install
$ steghide info image.jpg                        # Check if data hidden
$ steghide extract -sf image.jpg                 # Extract (asks password)
$ steghide extract -sf image.jpg -p ''           # Empty password
$ steghide extract -sf image.jpg -p 'password'   # With password
```

TOOL: stegcracker

Brute force steghide passwords.

```
$ pip install stegcracker
$ stegcracker image.jpg                          # Uses built-in wordlist
$ stegcracker image.jpg /path/to/rockyou.txt     # Custom wordlist
```

Python - Manual LSB Extraction

```
from PIL import Image

def extract_lsb(image_path, bits=1, channels='rgb', order='xy'):
    """Extract LSB data from image.
    bits: how many LSBs to extract (1-8)
    channels: which channels ('r', 'g', 'b', 'rgb', etc.)
    order: 'xy' (row by row) or 'yx' (column by column)
```

```
    """
    img = Image.open(image_path).convert('RGBA')
    w, h = img.size
    pixels = img.load()

    channel_map = {'r': 0, 'g': 1, 'b': 2, 'a': 3}
    binary = ''

    if order == 'xy':
        coords = [(x, y) for y in range(h) for x in range(w)]
    else:
        coords = [(x, y) for x in range(w) for y in range(h)]

    for x, y in coords:
        pixel = pixels[x, y]
        for ch in channels:
            idx = channel_map[ch]
            val = pixel[idx]
            binary += format(val, '08b')[-bits:]

    # Convert binary to bytes
    data = bytearray()
    for i in range(0, len(binary) - 7, 8):
        byte = int(binary[i:i+8], 2)
        data.append(byte)
        if byte == 0:  # null terminator
            break

    return bytes(data)

# Try different combinations
for ch in ['rgb', 'r', 'g', 'b']:
    for bits in [1, 2]:
        result = extract_lsb('image.png', bits=bits, channels=ch)
        printable = result.decode('ascii', errors='replace')[:100]
        if any(c.isalpha() for c in printable[:20]):
            print(f"bits={bits}, channels={ch}: {printable}")
```

StegSolve - Visual Bit Plane Analysis

[Difficulty: Easy-Medium]

TOOL: StegSolve

Java GUI for browsing image bit planes and color filters.

```
$ Download: github.com/Giotino/stegsolve
$ java -jar StegSolve.jar
```

WALKTHROUGH: Using StegSolve

Step 1: Step 1: Open image

```
File -> Open -> select image
```

Step 2: Step 2: Browse bit planes with arrow buttons (<, >)

```
Cycle through: Red 0, Red 1...Red 7, Green 0...Blue 7, Alpha...
```

Step 3: Step 3: Look for hidden patterns

```
Hidden QR codes often appear in Red plane 0 or Blue plane 0
Hidden text appears as black/white pixels in specific planes
```

Step 4: Step 4: Analyse -> Data Extract

```
Choose bits, channels, and byte order
Click 'Preview' to see extracted data
Click 'Save Bin' to save raw bytes
```

Step 5: Step 5: Analyse -> Frame Browser

```
For animated GIFs - browse individual frames
Flags sometimes hidden in specific frames
```

PNG Chunk Analysis & Dimension Tricks

[Difficulty: Medium]

PNG Structure

PNG files consist of chunks: IHDR (header with dimensions), IDAT (image data), tEXt (text comments), IEND (end marker). Each chunk has a CRC checksum.

WALKTHROUGH: PNG Hidden Dimensions Trick

Step 1: Common trick: IHDR height is reduced to hide bottom of image

```
Image appears normal but bottom portion is cropped
```

Step 2: Step 1: Check with pngcheck:

```
$ pngcheck -v image.png
If CRC error on IHDR -> dimensions were modified
```

Step 3: Step 2: Fix height by brute forcing correct value:

```
Try increasing height until CRC matches
OR: just set a very large height and recalculate CRC
```

Step 4: Step 3: Open fixed image to see hidden content

```
Flag or secret message visible in previously hidden area
```

Python - Fix PNG Dimensions

```
import struct, zlib
from PIL import Image
from io import BytesIO

with open('image.png', 'rb') as f:
    data = bytearray(f.read())

# PNG IHDR chunk is at bytes 8-29
# Width at bytes 16-19, Height at bytes 20-23
current_w = struct.unpack('>I', data[16:20])[0]
current_h = struct.unpack('>I', data[20:24])[0]
print(f"Current dimensions: {current_w}x{current_h}")

# Try different heights
for h in range(current_h, 3000):
    test = bytearray(data)
    test[20:24] = struct.pack('>I', h)
    # Recalculate IHDR CRC (over bytes 12-28: chunk type + data)
    crc = zlib.crc32(bytes(test[12:29]))
    test[29:33] = struct.pack('>I', crc & 0xffffffff)
    try:
        img = Image.open(BytesIO(bytes(test)))
        img.verify()
        print(f"Valid PNG at height {h}!")
        # Save fixed version
        with open('fixed.png', 'wb') as f:
            f.write(bytes(test))
        break
    except:
        pass
```

TOOL: pngcheck

Verify PNG file structure and checksums.

```
$ pngcheck -v image.png        # verbose check
$ pngcheck -7 image.png        # check for all issues
```

binwalk - Embedded File Extraction

[Difficulty: Easy]

TOOL: binwalk

Scan for and extract embedded files inside any binary.

```
$ sudo apt install binwalk                    # Install
```

WALKTHROUGH: Using binwalk

Step 1: Step 1: Scan for embedded files:

```
$ binwalk image.png
DECIMAL    HEXADECIMAL    DESCRIPTION
0          0x0            PNG image, 800x600
524288     0x80000        Zip archive data
524320     0x80020        End of Zip archive
```

Step 2: Step 2: Extract embedded files:

```
$ binwalk -e image.png
Creates: _image.png.extracted/ directory
Contains: 80000.zip (the embedded ZIP)
```

Step 3: Step 3: Examine extracted files:

```
$ ls _image.png.extracted/
80000.zip
$ unzip 80000.zip
flag.txt
```

Step 4: Step 4: If normal extract fails:

```
$ binwalk --dd='.*' image.png  # extract EVERYTHING
$ binwalk -E image.png  # entropy analysis (high entropy = encrypted)
```

Image Comparison / XOR

[Difficulty: Medium]

When given two similar images, XOR or diff them to reveal hidden differences.

WALKTHROUGH: Image XOR Comparison

Step 1: Scenario: Two images that look identical

```
But one has hidden data in pixel differences
```

Step 2: XOR them in Python:

```
See script below - XOR corresponding pixels
Differences show up as non-black pixels
```

Step 3: Use StegSolve:

```
Analyse -> Image Combiner
Open second image, select XOR mode
Differences become visible
```

Python - Image XOR / Diff

```
from PIL import Image, ImageChops
import numpy as np

img1 = np.array(Image.open("image1.png"))
img2 = np.array(Image.open("image2.png"))

# XOR images
xor_result = np.bitwise_xor(img1, img2)
# Enhance: any difference becomes white
enhanced = (xor_result > 0).astype(np.uint8) * 255
Image.fromarray(enhanced).save("xor_result.png")
print("Saved xor_result.png - differences shown in white")

# Alternative: ImageChops difference
```

```
diff = ImageChops.difference(Image.open("image1.png"), Image.open("image2.png"))
diff.save("diff_result.png")
```

Aperi'Solve - All-in-One Online Tool

[Difficulty: Easy]

WALKTHROUGH: Using Aperi'Solve

Step 1: Go to https://aperisolve.com/

Step 2: Upload your image

Step 3: It automatically runs:

- exiftool (metadata)
- binwalk (embedded files)
- zsteg (LSB analysis)
- steghide (with empty password)
- strings
- Color plane views

Step 4: Review all results on one page

The flag might be in any of these outputs

TIP: Aperi'Solve is the FASTEST way to check an image. Always try it first!

Chapter 9: Audio Steganography

Audio files hide data in spectrograms (visual), LSB of audio samples, metadata, or encoded signals like Morse and DTMF.

Spectrogram Analysis (Most Common!)

[Difficulty: Easy-Medium]

How It Works

An image or text is encoded as frequencies in an audio file. When you view the spectrogram (frequency vs time), the hidden image becomes visible. Very common in CTFs!

WALKTHROUGH: Spectrogram Analysis - Complete Walkthrough

Step 1: Step 1: Open audio in Audacity:

```
File -> Open -> select audio file
```

Step 2: Step 2: Switch to Spectrogram view:

```
Click the track name dropdown (left of waveform)
Select: Spectrogram
```

Step 3: Step 3: Adjust settings for better view:

```
Track dropdown -> Spectrogram Settings
- Window Size: 2048 or 4096
- Max Frequency: try 8000, 16000, 22050
- Color: Grayscale often works best
```

Step 4: Step 4: Look for hidden image/text:

```
Zoom out to see full spectrogram
Flags often appear as text written in the frequency domain
Example: 'FLAG{HIDDEN}' spelled out in the spectrogram
```

Step 5: Alternative: Use Sonic Visualiser:

```
Layer -> Add Spectrogram
Better rendering, more analysis options
```

TOOL: Audacity

Open-source audio editor with spectrogram view.

```
$ https://www.audacityteam.org/download/
$ Open file -> Track dropdown -> Spectrogram
```

TOOL: Sonic Visualiser

Advanced audio analysis with superior spectrogram rendering.

```
$ https://sonicvisualiser.org/
$ Layer -> Add Spectrogram -> adjust color and scale
```

Python - Generate Spectrogram Image

```
import numpy as np
from scipy.io import wavfile
import matplotlib.pyplot as plt

# Read audio file
rate, data = wavfile.read('audio.wav')
if len(data.shape) > 1:
    data = data[:, 0]  # mono (first channel)

# Generate spectrogram
plt.figure(figsize=(20, 8))
plt.specgram(data, Fs=rate, NFFT=1024, noverlap=512, cmap='gray')
plt.title('Spectrogram')
plt.xlabel('Time (seconds)')
plt.ylabel('Frequency (Hz)')
plt.colorbar(label='Intensity')
```

```
plt.savefig('spectrogram.png', dpi=200, bbox_inches='tight')
print("Saved spectrogram.png - look for hidden images/text!")
```

Audio LSB Steganography

[Difficulty: Medium]

How It Works

Similar to image LSB: data is hidden in the least significant bits of audio samples. Inaudible to human ears because the change is tiny (1 bit out of 16).

WALKTHROUGH: Audio LSB Extraction

Step 1: Step 1: Try stegolsb (WavSteg):

```
$ pip install stego-lsb
$ stegolsb wavsteg -r -i audio.wav -o output.txt -n 1 -b 10000
-n 1 = 1 LSB, -b 10000 = extract 10000 bytes
```

Step 2: Step 2: Check output.txt:

```
$ cat output.txt
flag{audio_lsb_steganography}
```

Step 3: Step 3: If stegolsb fails, try manual extraction:

```
Use Python script below to extract LSBs from WAV samples
```

Python - WAV LSB Extraction

```
import wave, struct

def extract_wav_lsb(wav_path, num_lsb=1):
    wav = wave.open(wav_path, 'r')
    n_frames = wav.getnframes()
    sample_width = wav.getsampwidth()
    frames = wav.readframes(n_frames)
    wav.close()

    # Parse samples based on sample width
    if sample_width == 1:
        samples = struct.unpack(f'{n_frames}B', frames)
    elif sample_width == 2:
        samples = struct.unpack(f'<{n_frames}h', frames)
    else:
        samples = struct.unpack(f'<{n_frames}i', frames)

    # Extract LSBs
    binary = ''
    for sample in samples:
        for bit in range(num_lsb):
            binary += str((sample >> bit) & 1)

    # Binary to bytes
    result = bytearray()
    for i in range(0, len(binary) - 7, 8):
        byte = int(binary[i:i+8], 2)
        if byte == 0: break
        result.append(byte)

    return bytes(result)

data = extract_wav_lsb('audio.wav', num_lsb=1)
print(f"Extracted: {data[:200]}")
```

Morse Code in Audio

[Difficulty: Easy]

WALKTHROUGH: Decoding Morse from Audio

Step 1: Step 1: Open in Audacity, view waveform

```
Short bursts = dots, Long bursts = dashes
```

Step 2: Step 2: Manually decode or use online tool:

```
https://morsecode.world/international/decoder/audio-decoder-adaptive.html
Upload audio -> auto-decodes Morse
```

Step 3: Step 3: If speed is wrong:

```
In Audacity: Effect -> Change Speed
Slow down or speed up to make beeps clearer
```

DTMF Tones (Phone Keypad Sounds)

[Difficulty: Easy-Medium]

DTMF tones represent phone keypad digits (0-9, *, #). Each key produces two specific frequencies.

WALKTHROUGH: Decoding DTMF from Audio

Step 1: Use multimon-ng:

```
$ multimon-ng -t wav -a DTMF audio.wav
DTMF: 1
DTMF: 2
DTMF: 3
Digits: 123 -> may need further decoding (phone keypad cipher?)
```

Step 2: Online alternative:

```
http://dtmf.netlify.app/ - Upload audio, auto-detects tones
```

Audio Metadata & Hidden Content

[Difficulty: Easy]

WALKTHROUGH: Checking Audio Files

Step 1: Step 1: Check metadata:

```
$ exiftool audio.mp3
$ mediainfo audio.mp3
$ ffprobe audio.mp3
```

Step 2: Step 2: Extract album art:

```
$ ffmpeg -i audio.mp3 -an -vcodec copy cover.jpg
The cover art image might contain a flag or more stego
```

Step 3: Step 3: Check for appended data:

```
$ binwalk audio.mp3
$ strings audio.mp3 | grep -i flag
```

Step 4: Step 4: Try playing backwards:

```
$ ffmpeg -i audio.wav -af areverse reversed.wav
Hidden messages sometimes in reversed audio
```

Step 5: Step 5: Try DeepSound (Windows):

```
DeepSound 2.0 - extract hidden files from audio
Try with empty password first
```

TOOL: DeepSound

Windows GUI tool for audio steganography.

```
$ Download: jpinsoft.net/DeepSound/
$ Open audio -> Extract Secret Files -> try empty password
```

Chapter 10: Text, Document & Network Steganography

Whitespace Steganography

[Difficulty: Easy-Medium]

Data hidden using invisible characters: trailing spaces, tabs at line ends, or zero-width Unicode characters mixed into text.

WALKTHROUGH: Whitespace Stego Detection & Extraction

Step 1: Step 1: Check for trailing whitespace:

```
$ cat -A message.txt  # shows $ at line ends, ^I for tabs
Look for spaces/tabs after the last visible character
```

Step 2: Step 2: Use snow/stegsnow:

```
$ stegsnow -C message.txt
Hidden message extracted from trailing whitespace
```

Step 3: Step 3: If password protected:

```
$ stegsnow -C -p 'password' message.txt
```

Step 4: Step 4: Check for zero-width characters:

```
Open in hex editor: look for E2 80 8B (ZWSP), E2 80 8C (ZWNJ)
Or paste text into: https://330k.github.io/misc_tools/unicode_steganography.html
```

TOOL: stegsnow / snow

Hide/extract data in whitespace.

```
$ sudo apt install stegsnow
$ stegsnow -C message.txt                  # extract without password
$ stegsnow -C -p 'password' message.txt    # extract with password
```

Zero-Width Character Steganography

[Difficulty: Medium]

WALKTHROUGH: Zero-Width Character Detection

Step 1: Clue: text file is larger than expected for its content

```
Or: copy-pasting text gives different results
```

Step 2: Step 1: Check file size vs visible character count:

```
$ wc -c file.txt  # byte count
$ wc -m file.txt  # character count
If byte count >> character count -> hidden Unicode chars
```

Step 3: Step 2: View in hex:

```
$ xxd file.txt | grep -E 'e280 (8b|8c|8d|a0)'
These are zero-width Unicode characters
```

Step 4: Step 3: Decode online:

```
https://330k.github.io/misc_tools/unicode_steganography.html
https://zwsp.netlify.app/
Paste text -> reveals hidden message
```

PDF Steganography

[Difficulty: Medium]

WALKTHROUGH: Analyzing PDFs for Hidden Data

Step 1: Step 1: Extract text:

```
$ pdftotext document.pdf output.txt
Check for hidden text layers (white text on white background)
```

Step 2: Step 2: Check metadata:

```
$ exiftool document.pdf
$ pdfinfo document.pdf
```

Step 3: Step 3: Scan for embedded files:

```
$ binwalk document.pdf
$ binwalk -e document.pdf
```

Step 4: Step 4: Check for JavaScript:

```
PDFs can contain JS code:
$ python3 peepdf.py document.pdf  # pip install peepdf
```

Step 5: Step 5: Extract images:

```
$ pdfimages -all document.pdf output_prefix
Images might contain stego data
```

Step 6: Step 6: Select all text (Ctrl+A):

```
In PDF viewer, select all -> paste into text editor
Reveals hidden/invisible text
```

Network / Protocol Steganography

[Difficulty: Hard]

Data hidden in network packets: DNS queries, ICMP payloads, TCP headers, HTTP headers, or packet timing.

WALKTHROUGH: PCAP Analysis for Hidden Data

Step 1: Step 1: Open in Wireshark:

```
$ wireshark capture.pcap
Look at Protocol column for unusual traffic
```

Step 2: Step 2: Check DNS exfiltration:

```
$ tshark -r capture.pcap -Y 'dns.qry.name' -T fields -e dns.qry.name
Domain names might contain encoded data:
example: ZmxhZw.evil.com, dGhpcw.evil.com -> base64 in subdomains
```

Step 3: Step 3: Check ICMP payloads:

```
$ tshark -r capture.pcap -Y 'icmp' -T fields -e data.data
ICMP echo requests may carry hidden data in payload
```

Step 4: Step 4: Extract files from HTTP:

```
Wireshark: File -> Export Objects -> HTTP
Saves all files transferred over HTTP
```

Step 5: Step 5: Follow TCP streams:

```
Right-click packet -> Follow -> TCP Stream
Read conversation contents
```

tshark Commands for PCAP Analysis

```
# Extract DNS query names
tshark -r capture.pcap -Y "dns.qry.name" -T fields -e dns.qry.name

# Extract ICMP data
tshark -r capture.pcap -Y "icmp" -T fields -e data.data

# Extract HTTP URIs
tshark -r capture.pcap -Y "http.request" -T fields -e http.request.uri

# Follow TCP stream 0
tshark -r capture.pcap -z "follow,tcp,ascii,0"

# Extract all HTTP objects
tshark -r capture.pcap --export-objects http,exported_files/

# Scapy for custom extraction
python3 -c "
from scapy.all import rdpcap
for pkt in rdpcap('capture.pcap'):
    if pkt.haslayer('ICMP') and pkt.haslayer('Raw'):
```

```
        print(bytes(pkt['Raw'].load))
"
```

Chapter 11: File Forensics & Carving

File forensics: analyzing file structures, recovering data, and extracting embedded content from binary files.

File Signature (Magic Bytes) Reference

[Difficulty: Easy]

File Type	Hex Signature	ASCII
PNG	89 50 4E 47 0D 0A 1A 0A	.PNG....
JPEG	FF D8 FF E0/E1	....
GIF	47 49 46 38 37/39 61	GIF87a/89a
PDF	25 50 44 46	%PDF
ZIP/DOCX	50 4B 03 04	PK..
RAR	52 61 72 21 1A 07	Rar!..
7z	37 7A BC AF 27 1C	7z....
ELF	7F 45 4C 46	.ELF
PE/EXE	4D 5A	MZ
BMP	42 4D	BM
WAV/AVI	52 49 46 46	RIFF
MP3	FF FB / 49 44 33	../ID3
GZIP	1F 8B	..

WALKTHROUGH: Fixing Corrupted File Headers

Step 1: Scenario: file command says 'data' instead of recognizing format

```
$ file mystery.bin
mystery.bin: data  <-- unrecognized
```

Step 2: Step 1: Check hex header:

```
$ xxd mystery.bin | head -3
00000000: 0050 4e47 0d0a 1a0a ...  .PNG....
First byte is 00 instead of 89 -> corrupted PNG header
```

Step 3: Step 2: Fix the header:

```
Correct PNG: 89 50 4E 47 0D 0A 1A 0A
python3 -c "
data = open('mystery.bin','rb').read()
data = b'\x89' + data[1:]  # fix first byte
open('fixed.png','wb').write(data)
"
```

Step 4: Step 3: Open fixed file:

```
$ file fixed.png
fixed.png: PNG image data, 800x600
```

strings - Extract Readable Text

[Difficulty: Easy]

WALKTHROUGH: Using strings Effectively

Step 1: Basic usage:

```
$ strings file.bin
Prints all printable character sequences (default min 4 chars)
```

Step 2: Search for flags/passwords:

```
$ strings file.bin | grep -iE 'flag|ctf|key|pass|secret'
```

```
flag{found_in_strings}
```

Step 3: Longer strings only:

```
$ strings -n 10 file.bin  # minimum 10 characters
Filters out noise
```

Step 4: Unicode/wide strings:

```
$ strings -e l file.bin  # 16-bit little-endian (Windows)
$ strings -e b file.bin  # 16-bit big-endian
```

ZIP / Archive Analysis & Cracking

[Difficulty: Easy-Medium]

WALKTHROUGH: ZIP Password Cracking

Step 1: Step 1: Check if password protected:

```
$ unzip -l archive.zip
$ 7z l archive.zip
```

Step 2: Step 2: Try common passwords:

```
Try: empty, 'password', challenge name, flag format
```

Step 3: Step 3: Brute force with fcrackzip:

```
$ fcrackzip -D -p /usr/share/wordlists/rockyou.txt -u archive.zip
PASSWORD FOUND: p = secret123
```

Step 4: Step 4: Or use John the Ripper:

```
$ zip2john archive.zip > zip_hash.txt
$ john zip_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
password123  (archive.zip)
```

Step 5: Step 5: If ZIP is corrupted:

```
$ zip -FF corrupted.zip --out fixed.zip
$ 7z x -p'' archive.zip  # 7z handles more corruption
```

Hex Editing & Analysis

[Difficulty: Easy-Medium]

WALKTHROUGH: Hex Analysis Workflow

Step 1: Step 1: View file header:

```
$ xxd file.bin | head -20
Identify file type from magic bytes
```

Step 2: Step 2: View file end:

```
$ xxd file.bin | tail -20
Check for appended data after file's end marker
```

Step 3: Step 3: Search for patterns:

```
$ xxd file.bin | grep '666c6167'  # 'flag' in hex
$ xxd file.bin | grep '504b0304'  # ZIP signature
```

Step 4: Step 4: Edit hex values:

```
$ xxd file.bin > hex.txt
# Edit hex.txt in text editor
$ xxd -r hex.txt > modified.bin
```

Step 5: GUI hex editors:

```
Windows: HxD (free), 010 Editor
Linux: bless, ghex
Online: hexed.it
```

Polyglot Files

[Difficulty: Medium-Hard]

A polyglot file is valid as multiple file types simultaneously. Example: a file that's both a JPEG and a ZIP.

WALKTHROUGH: Dealing with Polyglots

Step 1: Clue: binwalk shows multiple file signatures

```
$ binwalk polyglot.jpg
0        0x0       JPEG image
52000    0xCB20    Zip archive data
```

Step 2: Step 1: Try opening as different types:

```
Open as image: shows a picture
Rename to .zip and unzip: extracts hidden files!
```

Step 3: Step 2: Extract embedded files:

```
$ binwalk -e polyglot.jpg
OR: $ unzip polyglot.jpg (if it contains ZIP data)
```

Chapter 12: Methodology, Resources & Quick Reference

STEGO Challenge - Complete Decision Tree

[Difficulty: All Levels]

WALKTHROUGH: Universal Stego Methodology

Step 1: 1. file command

```
Identify TRUE file type (not just extension)
```

Step 2: 2. exiftool / metadata

```
Check ALL metadata fields for flags/hints
```

Step 3: 3. strings + grep

```
strings file | grep -iE 'flag|ctf|key|secret'
```

Step 4: 4. xxd | head

```
Check file header, verify magic bytes
```

Step 5: 5. binwalk / binwalk -e

```
Find and extract embedded files
```

Step 6: 6. PNG? -> zsteg -a, pngcheck

```
LSB analysis, chunk validation, dimension check
```

Step 7: 7. JPEG? -> steghide, stegcracker

```
Extract with empty password, then brute force
```

Step 8: 8. Image? -> StegSolve / Aperi'Solve

```
Bit plane analysis, color filters, online check
```

Step 9: 9. Audio? -> Audacity spectrogram

```
View spectrogram for hidden images/text
```

Step 10: 10. Audio? -> multimon-ng, stegolsb

```
DTMF detection, WAV LSB extraction
```

Step 11: 11. Text? -> stegsnow, zero-width check

```
Whitespace stego, Unicode invisible chars
```

Step 12: 12. Compare with original if provided

```
XOR/diff two images to find differences
```

Step 13: 13. Try common passwords

```
Empty, 'password', filename, challenge name
```

Step 14: 14. Check for data after file end marker

```
Data appended after PNG IEND or JPEG FFD9
```

CRYPTO Challenge - Complete Decision Tree

[Difficulty: All Levels]

WALKTHROUGH: Universal Crypto Methodology

Step 1: 1. Is it encoded? (Base64/Hex/Binary)

```
Try CyberChef Magic for auto-detection
```

Step 2: 2. Is it a known cipher?

```
dcode.fr/cipher-identifier / boxentriq cipher ID
```

Step 3: 3. Looks like shifted English?

```
Caesar brute force (25 shifts)
```

Step 4: 4. Looks random with patterns?

```
Vigenere (find key length with IoC)
```

Step 5: 5. Looks like random substitution?

```
quipqiup.com automatic solver
```

Step 6: 6. Given n, e, c numbers?

```
RSA! Try factordb -> RsaCtfTool
```

Step 7: 7. Given hex/binary data + key?
Try XOR, AES, DES decryption
Step 8: 8. XOR with unknown key?
Single-byte brute force, then xortool
Step 9: 9. Hash value? (32/40/64 hex chars)
hashid + crackstation.net + hashcat
Step 10: 10. AES/block cipher?
Check mode: ECB (repeated blocks), CBC (IV)
Step 11: 11. Still stuck?
Re-read challenge description for hints

Essential Tool Installation Guide

Linux (Kali/Ubuntu) - Install All Tools

```
# Crypto tools
pip install pycryptodome gmpy2 sympy z3-solver owiener
pip install factordb-pycli hashpumpy randcrack hashid
pip install xortool base58

# Stego tools
sudo apt install steghide binwalk foremost exiftool pngcheck
sudo apt install stegsnow sonic-visualiser audacity
pip install stego-lsb Pillow numpy scipy matplotlib
gem install zsteg

# Hash cracking
sudo apt install hashcat john fcrackzip

# Network
sudo apt install wireshark tshark
pip install scapy

# RSA
git clone https://github.com/RsaCtfTool/RsaCtfTool
cd RsaCtfTool && pip install -r requirements.txt
```

Python Libraries Quick Install

```
pip install pycryptodome   # AES, RSA, DES, hashing
pip install gmpy2          # Fast math (RSA cube roots, etc.)
pip install sympy          # Number theory, discrete_log
pip install z3-solver      # SAT/SMT constraint solver
pip install owiener        # Wiener's RSA attack
pip install factordb-pycli # FactorDB Python API
pip install hashpumpy      # Hash length extension
pip install randcrack      # MT19937 PRNG prediction
pip install xortool        # XOR key analysis
pip install base58         # Base58 encoding
pip install stego-lsb      # Audio LSB stego
pip install Pillow         # Image processing
pip install numpy scipy    # Math and signal processing
pip install matplotlib     # Spectrograms and plotting
```

Online Resources Reference

Resource	URL	Best For
CyberChef	gchq.github.io/CyberChef	Encoding/decoding
dcode.fr	dcode.fr	Cipher solving

quipqiup	quipqiup.com	Substitution ciphers
Aperi'Solve	aperisolve.com	Image stego (all-in-one)
FactorDB	factordb.com	RSA factoring
CrackStation	crackstation.net	Hash lookup
hexed.it	hexed.it	Online hex editor
Boxentriq	boxentriq.com/code-breaking	Cipher identification
cryptii	cryptii.com	Encoding conversion
Morse Decoder	morsecode.world	Audio Morse decode
Unicode Stego	330k.github.io/misc_tools	Zero-width stego
DTMF Decoder	dtmf.netlify.app	Phone tone decoding

CTF Practice Platforms

Platform	URL	Focus
PicoCTF	picoctf.org	Beginner-friendly
CryptoHack	cryptohack.org	Crypto challenges
CryptoPals	cryptopals.com	Crypto exercises
OverTheWire	overthewire.org	Linux/security basics
HackTheBox	hackthebox.com	Medium-Hard boxes
TryHackMe	tryhackme.com	Guided learning
Root-Me	root-me.org	All categories
CTFtime	ctftime.org	Competition calendar

TABLE OF CONTENTS